CS683 PA2: Summary

Task 1: Stride Pattern Prefetcher (L1D); Task 2A: Instruction-conscious L2 replacement (I-RRIP); Task 2B: Dynamic I-RRIP; Task 3: SPP + prefetch-aware Dynamic I-RRIP

Task 1A: IP-stride (baseline prefetcher)

`ip_stride.l1d_pref` keeps 24 IP trackers (IP, last line address, last stride, LRU). On an access by a tracked IP it computes the stride from the last line; if it equals the stored stride it prefetches the next 3 strides ahead (same page only), otherwise it just stores the new stride. It therefore captures only a *constant* stride and is blind to repeating non-uniform sequences such as $(1, 2, 3)^n$. It is tracked by a fully-associative LRU table of 24 entries, so IPs beyond 24 thrash it.

Task 1B: SPP design

- **IP table** (256 entries, direct mapped, 8-bit tag): last line (low 16 bits), last stride, a 2-bit uniform-stride confidence, and a signature of the last 3 strides (7 bits each).
- **Pattern table** (256 entries) indexed by a hash of the 3-stride signature → (next stride, 2-bit confidence). A **fallback table** (64 entries) is keyed by the last 2 strides and is used when the 3-stride entry is not confident.
- **Prediction**: if the uniform confidence is high the constant stride is used; otherwise the pattern table is walked, feeding each predicted stride back into the signature, up to the current degree. Prefetching stops at the 4KB page boundary and when the prefetch queue is full. Only loads/RFOs train. Prefetches fill L1D unless the MSHR is more than half full, then L2.
- **Accuracy throttling**: a 64-entry filter of recently filled prefetch lines; a demand hit on it counts as useful. Every 256 prefetch fills accuracy is computed and the degree set: ≥75%→6, ≥50%→4, ≥25%→2, else 1 (initial 4).

Results (25M warmup + 25M simulated instructions)

Speedup = IPC / baseline IPC (mean = geometric). Coverage = (baseline L1D load misses − misses with prefetcher)/baseline. Accuracy is the simulator's (useful / prefetches issued to the lower level). **IPC**

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	0.2323	0.2512	0.3821	0.2885
IP-stride	0.2335	0.2513	0.3829	0.2892
SPP (ours)	0.2343	0.2549	0.3872	0.2921

Speedup

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	1.0000	1.0000	1.0000	1.0000
IP-stride	1.0048	1.0004	1.0021	1.0024
SPP (ours)	1.0082	1.0149	1.0135	1.0122

L1D load MPKI

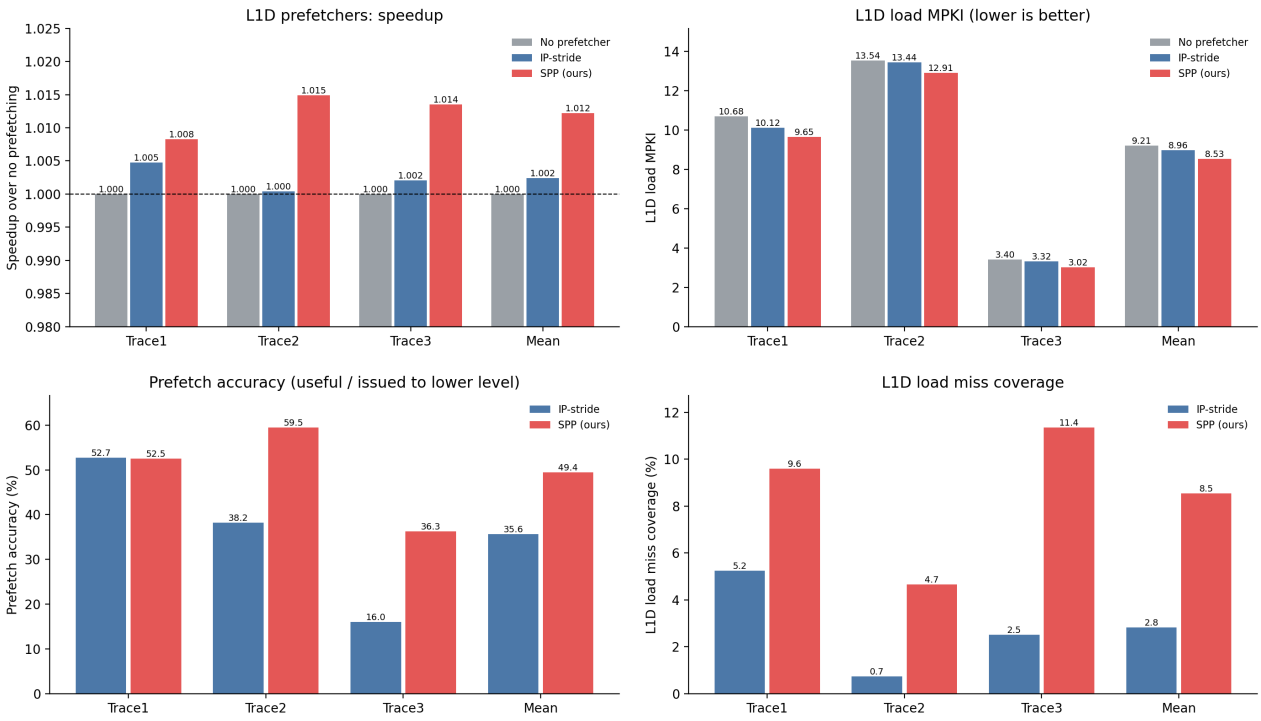
Config	Trace1	Trace2	Trace3	Mean
No prefetcher	10.678	13.538	3.403	9.206
IP-stride	10.119	13.438	3.318	8.958
SPP (ours)	9.653	12.907	3.017	8.526

Prefetch accuracy (%)

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	n/a	n/a	n/a	n/a
IP-stride	52.71	38.18	15.99	35.63
SPP (ours)	52.49	59.53	36.27	49.43

L1D load coverage (%)

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	n/a	n/a	n/a	n/a
IP-stride	5.24	0.74	2.51	2.83
SPP (ours)	9.60	4.66	11.36	8.54



SPP beats IP-stride on every trace in speedup and coverage (mean speedup 1.0122 vs 1.0024; coverage 8.5% vs 2.8%), and has higher accuracy on Trace 2 and 3 (tied on Trace 1). Gains are small in absolute terms: a miss breakdown showed most L1D misses come from IPs not in the table, strides beyond a page (forbidden by the spec) or irregular accesses; only about 3 to 5% of misses occur while a stride pattern is active.

Storage overhead (exact)

Structure	Entries	Bits per entry	Total bits
IP table	256	valid 1 + tag 8 + last line 16 + last stride 7 + uni conf 2 + 3×7 signature + hist count 2 = 57	14592
Pattern table	256	stride 7 + conf 2 = 9	2304
Fallback table	64	stride 7 + conf 2 = 9	576
Recent-prefetch filter	64	valid 1 + tag 10 = 11	704
Counters	n/a	issued 9 + useful 9 + degree 3	21
Total		2274.6 B ≈ 2.2 KB	**18197**

For comparison, the provided IP-stride uses 24 × (64+64+64+32) = 5376 bits as coded; SPP uses about 3.4× more storage.

Threshold and degree rationale

Accuracy thresholds 75/50/25% divide accuracy into four bands: above 75% prefetches are almost always used, so aggressive depth (6) is cheap; 50 to 75% is the normal band (4); 25 to 50% means over half the prefetches are wasted, so depth is halved (2); below 25% prefetching costs more bandwidth and L1D pollution than it saves, so only one line is issued. The values are round quartile boundaries; I validated them by sweeping: thresholds 50/30/10, 60/40/20, 90/70/40 and degree sets (8,4,2,1), (8,6,4,2) all lie within ±0.1% of mean speedup (1.0115 to 1.0129), so the result is insensitive to them. Fixed degrees 1/2/4/6 give mean speedups 1.0076/1.0091/1.0101/1.0102, so deeper is better up to 4, and flat beyond. **Honest finding:** dynamic throttling gives the same speedup as fixed degree 4 (1.0122 vs 1.0124) but 4 to 9 points higher accuracy, i.e. it saves bandwidth without losing performance.

Ablations and design validation

Configuration	Mean speedup	Note
Uniform stride only (no pattern table)	1.0043	≈ IP-stride-like
Pattern table, no fallback	1.0100	
Pattern table + fallback (final)	1.0122	

IP-table size (without fallback): 64/128/256/512/1024 entries → 1.0054/1.0078/1.0100/1.0107/1.0111 (6.7/10.3/17.6/32.2/61.4 Kbit); 256 chosen as the knee. A single 2-stride table alone reached 1.0126, within noise of the final hybrid; the hybrid was kept because the 3-stride signature disambiguates longer patterns. Synthetic tests (uniform, non-uniform, phase changes, random, interleaved IPs) give about 97% coverage on regular streams, about 0% on random, and 0 page-boundary violations in 186K prefetches; the one weakness is interleaved IPs sharing a signature in the pattern table (78%), a known trade-off since IP bits in the index lowered real-trace speedup.

Task 2A: Instruction-conscious L2 replacement (I-RRIP)

All Task 2 runs use no L1D prefetcher, so the replacement policy is evaluated in isolation (Task 3 combines it with SPP). The baseline is LRU at L2.

Exposing instruction information. `update_replacement_state` was extended with a 9th parameter `isInstruction` for every cache (declarations in `inc/cache.h`, all `replacement/*_repl` definitions). The six call sites in `src/cache.cc` pass the packet's `instruction` field (MSHR entry on fills, RQ/WQ/PQ entry on hits); the BTB calls in `src/ooo_cpu.cc` pass 0. For eviction, the policy reads the per-line `BLOCK::instruction` bit, which `fill_cache` already sets for L2C lines filled by instruction loads/prefetches, through the `current_set` argument of `find_victim`.

Policy (`replacement/irrip.l2c_repl`, 2-bit RRPV as in SRRIP):

- **Insertion:** instruction lines at RRPV 1, data lines at RRPV 3 (SRRIP: both at 2).

- **Promotion:** an instruction hit sets RRPV to 0; a data hit decrements RRPV by 1 (SRRIP: both to 0).
- **Eviction:** among lines at RRPV 3, a data line is evicted before an instruction line; aging is unchanged.

Results (25M warmup + 25M simulation instructions; speedup is geometric mean over the three traces). **IPC**

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	0.2322	0.2513	0.3819	0.2885
SRRIP	0.2322	0.2518	0.3816	0.2885
I-RRIP	0.2319	0.2597	0.3826	0.2914

Speedup over baseline

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	1.0000	1.0000	1.0000	1.0000
SRRIP	0.9999	1.0022	0.9993	1.0004
I-RRIP	0.9986	1.0334	1.0020	1.0112

L2C load MPKI

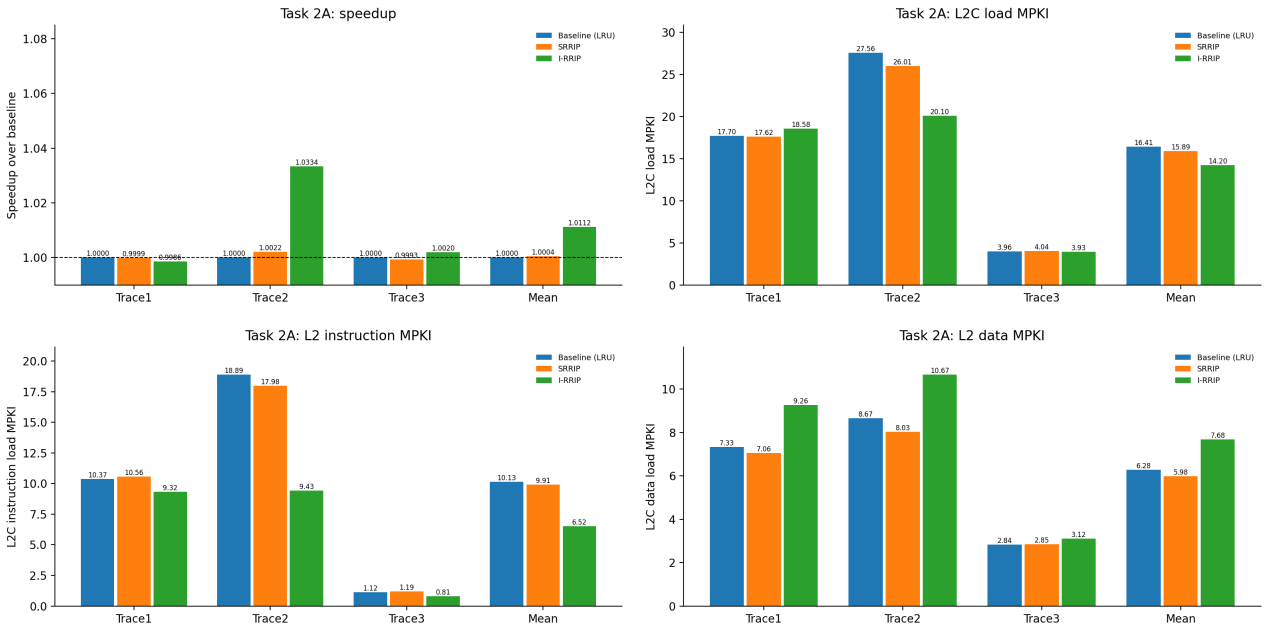
Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	17.703	27.560	3.960	16.408
SRRIP	17.619	26.012	4.037	15.889
I-RRIP	18.578	20.104	3.928	14.204

L2C instruction load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	10.368	18.892	1.124	10.128
SRRIP	10.563	17.980	1.190	9.911
I-RRIP	9.316	9.435	0.811	6.520

L2C data load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	7.334	8.668	2.837	6.280
SRRIP	7.056	8.032	2.846	5.978
I-RRIP	9.263	10.669	3.118	7.683

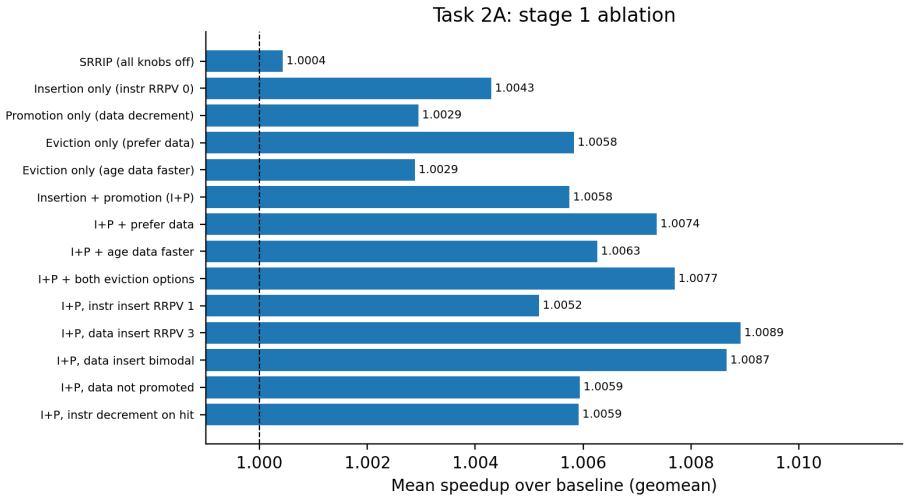


Findings. I-RRIP reaches a mean speedup of 1.0112, against 1.0004 for SRRIP. It cuts the mean L2 instruction load MPKI from 9.91 (SRRIP) to 6.52 (−34%), at the cost of data load MPKI rising from 5.98 to 7.68 (+28%); total L2 load MPKI still falls 11% (15.89 to 14.20), because instruction misses are both the larger share and the more costly ones (an instruction miss stalls the front end, while out-of-order execution hides part of a data miss). The gain is concentrated on Trace 2, the trace with by far the highest L2 instruction MPKI: instruction MPKI almost halves (17.98 to 9.44) and IPC improves 3.3%. Trace 3 gains 0.2%. Trace 1 loses 0.14%: its LLC misses about 69% of the loads it receives under every policy, so the extra L2 data misses go to DRAM, and they outnumber the instruction misses saved (data MPKI +2.21 versus instruction MPKI −1.25 relative to SRRIP). Blindly prioritising instructions is therefore not always right, which is the motivation for Task 2B.

How the design was chosen. Every part of the policy was made a compile-time knob: instruction insertion RRPV (0/1/2), data insertion (RRPV 2 / RRPV 3 / bimodal: RRPV 3 with every 32nd insertion at 2), instruction promotion (to 0 / decrement), data promotion (to 0 / decrement / unchanged), and two eviction options (prefer a data victim among RRPV-3 lines; age data lines by 2 instead of 1). The full grid is $3 \cdot 3 \cdot 2 \cdot 3 \cdot 2 \cdot 2 = 216$ configurations, so the search was done in two stages. *Stage 1* (14 configurations) turned on one change at a time, both on top of SRRIP (an ablation of insertion, promotion and eviction) and on top of I+P, the combination of instruction insertion at RRPV 0 (instruction hits to 0) and data decrement on hit. The all-SRRIP setting reproduces `rrip.l2c_repl` exactly, which validates the knob implementation.

Stage 1: speedup over baseline

Configuration	Trace1	Trace2	Trace3	Mean
SRRIP (all knobs off)	0.9999	1.0022	0.9993	1.0004
Insertion only (instr RRPV 0)	1.0000	1.0130	0.9999	1.0043
Promotion only (data decrement)	1.0008	1.0085	0.9995	1.0029
Eviction only (prefer data)	0.9998	1.0185	0.9993	1.0058
Eviction only (age data faster)	1.0004	1.0088	0.9994	1.0029
Insertion + promotion (I+P)	0.9998	1.0176	0.9999	1.0058
I+P + prefer data	0.9994	1.0229	1.0000	1.0074
I+P + age data faster	0.9993	1.0196	1.0001	1.0063
I+P + both eviction options	0.9983	1.0251	1.0000	1.0077
I+P, instr insert RRPV 1	1.0003	1.0154	0.9999	1.0052
I+P, data insert RRPV 3	0.9968	1.0291	1.0013	1.0089
I+P, data insert bimodal	0.9971	1.0283	1.0008	1.0087
I+P, data not promoted	0.9978	1.0202	1.0000	1.0059
I+P, instr decrement on hit	1.0001	1.0176	1.0001	1.0059



Stage 2 took the knobs that changed the mean speedup by at least 0.05% in stage 1 (instruction insertion, data insertion, both eviction options), each at its two best values, and ran their full factorial ($2^4 = 16$ configurations). Promotion was fixed at instruction-to-0 and data-decrement: the instruction options tied within 0.01%, data-to-0

was clearly worse, and data-unchanged tied with decrement but is unsafe with data inserted at RRPV 3 (see below). Two stage-2 points repeat stage-1 points and reproduced bit-exactly.

Stage 2: speedup over baseline

Instr insert / data insert / prefer data / age data	Trace1	Trace2	Trace3	Mean
instr 1 / data RRPV 3 / prefer on / age on	0.9974	1.0348	1.0020	1.0113
instr 1 / data RRPV 3 / prefer on / age off (chosen)	0.9986	1.0334	1.0020	1.0112
instr 1 / data bimodal / prefer on / age on	0.9980	1.0343	1.0015	1.0111
instr 1 / data RRPV 3 / prefer off / age on	0.9982	1.0337	1.0018	1.0111
instr 1 / data bimodal / prefer on / age off	0.9990	1.0332	1.0014	1.0111
instr 1 / data bimodal / prefer off / age on	0.9986	1.0327	1.0012	1.0107
instr 1 / data RRPV 3 / prefer off / age off	0.9993	1.0313	1.0017	1.0107
instr 1 / data bimodal / prefer off / age off	0.9998	1.0305	1.0012	1.0104
instr 0 / data RRPV 3 / prefer on / age off	0.9961	1.0300	1.0014	1.0091
instr 0 / data RRPV 3 / prefer off / age off	0.9968	1.0291	1.0013	1.0089
instr 0 / data bimodal / prefer on / age on	0.9954	1.0299	1.0011	1.0087
instr 0 / data bimodal / prefer off / age on	0.9963	1.0291	1.0011	1.0087
instr 0 / data bimodal / prefer on / age off	0.9964	1.0290	1.0010	1.0087
instr 0 / data bimodal / prefer off / age off	0.9971	1.0283	1.0008	1.0087
instr 0 / data RRPV 3 / prefer off / age on	0.9955	1.0297	1.0011	1.0087
instr 0 / data RRPV 3 / prefer on / age on	0.9946	1.0300	1.0014	1.0085

Rationale: insertion (modified). Insertion decides how long a line survives before it has proved itself, so it is the most direct lever for prioritising instructions. Code lines are reused across loop iterations and repeated calls, but often at reuse distances larger than the L1I. Inserting data at RRPV 3 makes a new data line the first eviction candidate unless it is reused; this was the largest single improvement in stage 1 (1.0058 to 1.0089), which indicates that most data lines reaching L2 in these traces are not reused before they would be evicted anyway. Instructions are inserted at RRPV 1 rather than 0. This is the one result that only the full factorial revealed: with data inserted at RRPV 2, instruction RRPV 0 was slightly better (1.0058 versus 1.0052), but once data is inserted at RRPV 3, RRPV 1 is better in all eight pairings, by 0.17–0.28%. Data at RRPV 3 already gives code a two-step lead, so RRPV 0 is no longer needed to protect code from data; it only lets code that is fetched once (cold paths, one-time initialisation) push out code that is actually reused. Bimodal data insertion was within 0.04% of RRPV 3 in every stage-2 pairing (and never better) but adds a counter, so plain RRPV 3 was kept.

Rationale: promotion (modified for data only). An instruction hit is strong evidence of a hot code path, so it keeps SRRIP's promote-to-0 rule. The decrement alternative tied (1.0059 versus 1.0058), so the simpler and standard rule was kept. Data uses frequency-based promotion: one hit moves a line one step nearer, so only repeatedly reused data earns the protection code gets on its first hit. Promoting data to 0 on a hit would undo the insertion bias after a single reuse; measured against decrementing, it lowered the mean speedup from 1.0058 to 1.0043. Leaving data unpromoted tied with decrement on its own (1.0059), but combined with insertion at RRPV 3 a reused data line could never leave the eviction position, so decrement is the safe choice.

Rationale: eviction (modified as a tie-breaker only). SRRIP evicts the first line it finds at RRPV 3. Preferring a data line among the RRPV-3 candidates costs nothing extra, because the instruction bit is already stored per line. It only matters when code and data are equally old, and then evicting the data line is consistent with the rest of the policy. On its own it gave the largest gain of any single change on top of SRRIP (1.0004 to 1.0058), and in stage 2 it added about 0.05%. Aging data faster was *not* adopted: on top of the chosen insertion and promotion it moved the mean speedup by at most 0.01% (1.0112 versus 1.0113), but it raised the Trace 1 loss from 0.14% to 0.26% and needs a second aging rule. The aging procedure is therefore left as in SRRIP.

Storage. Per line: 2-bit RRPV (as SRRIP) and 1 instruction bit, which ChampSim's L2C already keeps in `BLOCK`. For 1024 sets × 8 ways this is 16384 + 8192 = 24576 bits (3 KB), of which only the 8192 instruction bits (1 KB) are on top of SRRIP. No counters or tables are added.

Task 2B: Dynamic I-RRIP

Same setup as Task 2A (no L1D prefetcher, LRU baseline, 25M warmup + 25M simulation instructions).

Policy (`replacement/dyn_irrip.l2c_repl`; the prefetch rules added in Task 3 never trigger here, since no prefetcher is used): set dueling between SRRIP and I-RRIP.

- **Leader sets:** 32 sets always run SRRIP and 32 always run I-RRIP (64 of the 1024 sets, 6.25%). The cache is split into 32 regions of 32 sets; in region r the set at offset r is the SRRIP leader and the set at offset $31 - r$ (the complement) is the I-RRIP leader, so both groups are spread over the whole cache and never overlap. Identifying a leader is a 5-bit compare on the set index; no table is needed.
- **Selector:** a 10-bit saturating counter PSEL, initialised to 512. A demand load miss in an SRRIP leader increments it, one in an I-RRIP leader decrements it. Follower sets (the other 960) run I-RRIP when the MSB of PSEL is 1 (the SRRIP leaders are missing more) and SRRIP otherwise.
- **SRRIP mode:** insert at RRPV 2, promote to 0 on a hit, evict the first line at RRPV 3.
- **I-RRIP mode:** instruction lines inserted at RRPV 1 and promoted to 0 on a hit; data lines inserted at RRPV 3 and decremented by 1 on a hit; among lines at RRPV 3 a data line is evicted first; when no line is at RRPV 3, data lines age by 2 and instruction lines by 1.

Results. "I-RRIP mode, static" is the I-RRIP mode above run on every set, to separate the effect of the selector from that of the mode itself. **IPC**

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	0.2322	0.2513	0.3819	0.2885
SRRIP	0.2322	0.2518	0.3816	0.2885
I-RRIP (Task 2A)	0.2319	0.2597	0.3826	0.2914
I-RRIP mode, static	0.2317	0.2600	0.3826	0.2914
Dynamic I-RRIP	0.2321	0.2597	0.3825	0.2914

Speedup over baseline

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	1.0000	1.0000	1.0000	1.0000
SRRIP	0.9999	1.0022	0.9993	1.0004
I-RRIP (Task 2A)	0.9986	1.0334	1.0020	1.0112
I-RRIP mode, static	0.9974	1.0348	1.0020	1.0113
Dynamic I-RRIP	0.9993	1.0335	1.0017	1.0114

L2C load MPKI

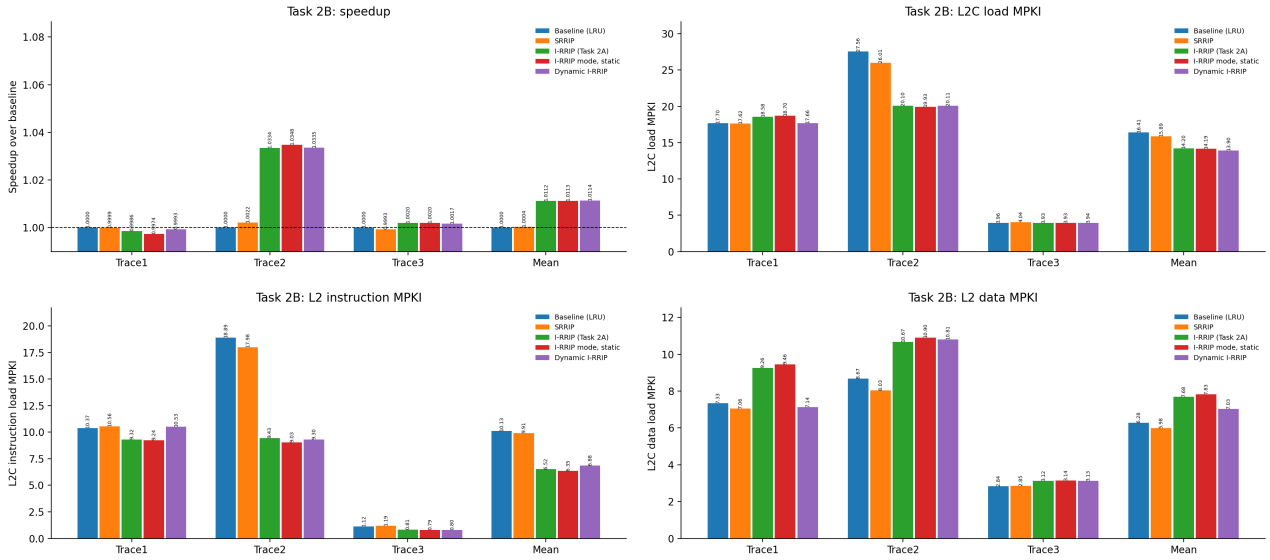
Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	17.703	27.560	3.960	16.408
SRRIP	17.619	26.012	4.037	15.889
I-RRIP (Task 2A)	18.578	20.104	3.928	14.204
I-RRIP mode, static	18.700	19.927	3.931	14.186
Dynamic I-RRIP	17.665	20.108	3.937	13.903

L2C instruction load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	10.368	18.892	1.124	10.128
SRRIP	10.563	17.980	1.190	9.911
I-RRIP (Task 2A)	9.316	9.435	0.811	6.520
I-RRIP mode, static	9.242	9.029	0.792	6.354
Dynamic I-RRIP	10.528	9.296	0.804	6.876

L2C data load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	7.334	8.668	2.837	6.280
SRRIP	7.056	8.032	2.846	5.978
I-RRIP (Task 2A)	9.263	10.669	3.118	7.683
I-RRIP mode, static	9.459	10.897	3.139	7.832
Dynamic I-RRIP	7.137	10.811	3.133	7.027



Selector behaviour (after warmup)

Trace	Leader demand misses SRRIP	I-RRIP	Final PSEL (of 1023)	Followers in I-RRIP mode	Mode switches
Trace1	13403	15120	0	0.0%	0
Trace2	19435	16414	1017	100.0%	0
Trace3	3202	3006	1020	100.0%	0

Findings. The selector makes the right choice on every trace and never changes its mind after warmup. On Trace 1 the I-RRIP leaders miss 13% more than the SRRIP leaders, PSEL sits at 0, and all follower sets run SRRIP: the Trace 1 loss shrinks from 0.26% (I-RRIP mode, static) to 0.07%, and data MPKI returns to the SRRIP level (7.14 versus 7.06). On Traces 2 and 3 the SRRIP leaders miss more (18% and 6%), PSEL saturates near 1023, and all followers run I-RRIP. Dynamic I-RRIP has the highest mean speedup of all policies (1.0114) and the lowest mean L2 load MPKI (13.90, against 14.20 for Task 2A's I-RRIP and 15.89 for SRRIP). The upper bound for any selector between these two modes, the better static mode on each trace, is 1.0121; the remaining 0.07% is the cost of the leader sets, which keep running the losing policy (on Trace 2, the 32 SRRIP leaders lower the speedup from 1.0348 to 1.0335).

Choice of the I-RRIP mode. Task 2A and Task 2B use different I-RRIP settings, because the two tasks are asked different questions. In Task 2A the policy is static: whatever it does to a trace, that trace keeps it. So it was chosen on all three traces, and the setting without faster data aging was kept. Its mean speedup is within noise of the best one (1.0112 versus 1.0113), and it loses less on Trace 1 (0.14% versus 0.26%).

In Task 2B the I-RRIP rules are only one of two options. Set dueling falls back to SRRIP on a workload where they hurt. Trace 1 is such a workload: replacement barely matters there (SRRIP 0.9999 against LRU), and every I-RRIP setting was slightly below the baseline. For Trace 1 the dynamic policy should therefore pick SRRIP, not a gentler I-RRIP, which is what the selector table shows. The I-RRIP mode only needs to be strong on the workloads where it gets picked. Trace 2 is the case the problem statement describes: a large code footprint, with the highest L2 instruction MPKI under SRRIP (17.98). The I-RRIP mode is therefore tuned for Trace 2. Adding faster data aging raises Trace 2's speedup from 1.0334 to 1.0348 and cuts its instruction MPKI from 9.43 to 9.03. The cost is one extra aging rule, used only in sets running I-RRIP.

How the design was chosen. The number of leader sets, their placement and the PSEL width were made compile-time knobs. Two sanity runs force all follower sets into one mode; they reproduce static SRRIP and the static I-RRIP mode to within 0.05% on every trace (the remaining difference is the leader sets of the other policy), which validates the mode switch.

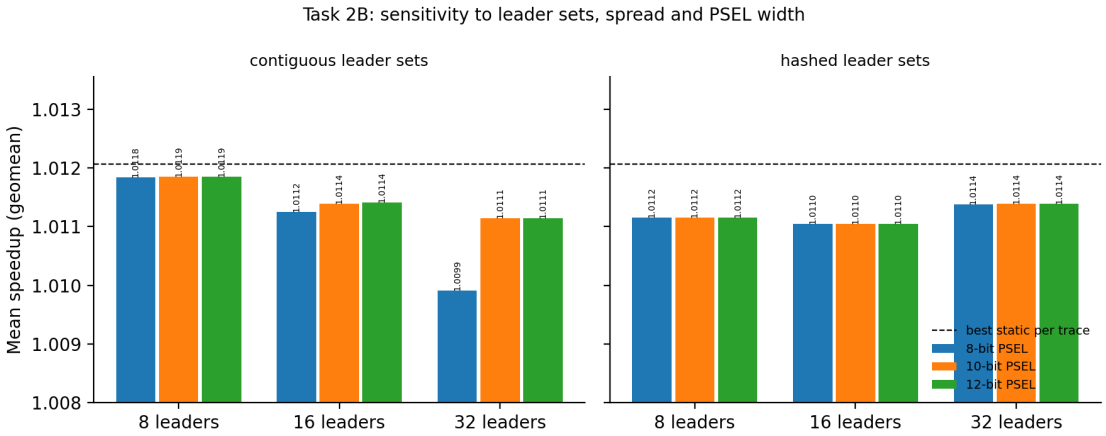
Sanity check: speedup over baseline

Configuration	Trace1	Trace2	Trace3	Mean
Followers forced to SRRIP	0.9997	1.0025	0.9993	1.0005
Static SRRIP	0.9999	1.0022	0.9993	1.0004
Followers forced to I-RRIP	0.9975	1.0343	1.0019	1.0111
Static I-RRIP mode	0.9974	1.0348	1.0020	1.0113

The sensitivity study then ran the full grid of 8/16/32 leader sets per policy, contiguous (sets $0..N-1$ lead SRRIP, $N..2N-1$ lead I-RRIP) or spread as above, and 8/10/12-bit PSEL (18 configurations). The last column is the share of Trace 1 follower updates done in I-RRIP mode; the right choice for Trace 1 is 0%. All configurations choose I-RRIP on Traces 2 and 3.

Sensitivity study: speedup over baseline

Leader sets / spread / PSEL	Trace1	Trace2	Trace3	Mean	Trace1 I-RRIP
8 / contiguous / 8-bit	0.9997	1.0343	1.0019	1.0118	6.6%
8 / contiguous / 10-bit	0.9998	1.0343	1.0019	1.0119	0.0%
8 / contiguous / 12-bit	0.9998	1.0343	1.0019	1.0119	0.0%
8 / hashed / 8-bit	0.9973	1.0346	1.0020	1.0112	100.0%
8 / hashed / 10-bit	0.9973	1.0346	1.0020	1.0112	100.0%
8 / hashed / 12-bit	0.9973	1.0346	1.0020	1.0112	100.0%
16 / contiguous / 8-bit	0.9994	1.0328	1.0019	1.0112	23.4%
16 / contiguous / 10-bit	0.9999	1.0328	1.0019	1.0114	2.5%
16 / contiguous / 12-bit	0.9999	1.0328	1.0019	1.0114	0.0%
16 / hashed / 8-bit	0.9974	1.0343	1.0019	1.0110	100.0%
16 / hashed / 10-bit	0.9974	1.0343	1.0019	1.0110	100.0%
16 / hashed / 12-bit	0.9974	1.0343	1.0019	1.0110	100.0%
32 / contiguous / 8-bit	0.9986	1.0329	0.9986	1.0099	58.6%
32 / contiguous / 10-bit	0.9990	1.0329	1.0019	1.0111	60.3%
32 / contiguous / 12-bit	0.9990	1.0329	1.0019	1.0111	60.3%
32 / hashed / 8-bit	0.9993	1.0335	1.0017	1.0114	2.6%
32 / hashed / 10-bit (chosen)	0.9993	1.0335	1.0017	1.0114	0.0%
32 / hashed / 12-bit	0.9993	1.0335	1.0017	1.0114	0.0%
Best static per trace	0.9999	1.0348	1.0020	1.0121	



Rationale: number and placement of leader sets. The leader sets are a sample of the cache, and the selector is only as good as the sample. Traces 2 and 3 are easy: every configuration picks I-RRIP. Trace 1 is the hard case, because the two modes differ there by only 0.25%, so the miss counts of the two leader groups are close and the outcome depends on which sets are sampled. With 8 or 16 leaders per policy the answer depends on placement: contiguous leaders choose SRRIP, while spread leaders choose I-RRIP. Since the same number of leaders gives opposite answers, these samples are too small to be trusted. With 32 spread leaders the choice is SRRIP, while

32 contiguous leaders flip back and forth (60% of the time in I-RRIP mode, 77 switches): sets 0..63 are a narrow, unrepresentative slice of the address space, and spreading the leaders over all regions avoids that. 8 contiguous leaders had the highest mean (1.0119 versus 1.0114), but only because that particular small sample happened to choose correctly on Trace 1; 32 spread leaders were chosen because their Trace 1 choice does not depend on luck. The price is that more sets run the losing policy: on Trace 2, speedup goes from 1.0343–1.0346 with 8 leaders to 1.0335 with 32. Going beyond 32 would raise this cost further (more than 6% of the cache would be fixed to the wrong policy) for a choice that is already stable.

Rationale: PSEL width. The width sets how many net misses are needed to change mode (half the range: 128 for 8 bits, 512 for 10, 2048 for 12). An 8-bit PSEL is too sensitive: with 32 spread leaders it still put 2.6% of Trace 1 follower updates in I-RRIP mode, and with 32 contiguous leaders it switched 87 times on Trace 1 and 25 times on Trace 3, where it cost 0.33%. At 10 and 12 bits the speedups are identical for every leader configuration (the only difference is 16 contiguous leaders spending 2.5% of Trace 1 in I-RRIP mode at 10 bits, with no effect on IPC), so 10 bits is the smallest width at which the decision is stable.

Rationale: decision logic. The selector compares the two policies directly, by the misses they actually cause in their leader sets, instead of predicting from the instruction share of accesses which policy should be better. This is what the problem asks for (reduce overall misses rather than favour instructions blindly), and it needs no threshold to tune: the decision follows from the outcome. Only demand load misses update PSEL, because they are the misses that stall the core; RFO, prefetch and writeback misses are mostly off the critical path and would bias PSEL toward a policy that reduces traffic rather than stalls. Instruction and data misses are weighted equally; a different weighting was not explored.

Storage. On top of Task 2A's per-line state (2-bit RRPV and the instruction bit, 24576 bits for 1024 sets × 8 ways), only the 10-bit PSEL is added: 24586 bits in total. Leader identification is combinational logic on the set index. The other counters in the code (follower updates, mode switches, leader misses) are simulation statistics and not part of the design.

Task 3: SPP with prefetch-aware Dynamic I-RRIP

Policy (`replacement/dyn_irrip.l2c_repl` with `PF_AWARE` 1; `PF_AWARE` 0 gives the Task 2B policy). The Task 2B policy is kept unchanged (32 SRRIP and 32 I-RRIP leader sets, 10-bit PSEL), and two prefetch rules are added to the I-RRIP mode only, so the SRRIP leader sets still represent plain SRRIP in the duel. A prefetched data line is inserted at RRPV 3 (first eviction candidate), and a prefetch request that hits a resident line does not promote it. Everything else is the Task 2B I-RRIP mode: instructions inserted at RRPV 1 and promoted to 0; demand data inserted at RRPV 3 and decremented by 1 on a hit, so a prefetched line that is actually used moves up; among RRPV-3 lines a data line is evicted first, and data ages by 2. Because the Task 2B I-RRIP mode already inserts all data at RRPV 3, the insertion rule places prefetches at the same position as demand data; the prefetch-specific difference is that prefetch hits do not promote.

Rationale. SPP brings lines into L1D, so the L2 copy of a prefetched line is a duplicate that is only needed if the L1D copy is evicted before use; a speculative line should therefore not displace demand data or instruction lines, which the L2 is supposed to protect. A prefetch hit is not evidence of reuse (the prefetcher re-requests a line it already fetched), so promoting on it would keep stale lines alive. Inserting at the distant position lets a useless prefetch leave quickly, while a demand hit still rescues a useful one. The prefetch insertion RRPV was swept (1/2/3); RRPV 3 is best on mean speedup (1.0236 against 1.0223 and 1.0225), and inserting prefetches closer than demand data raises the L2 instruction MPKI (7.25 at RRPV 3, 7.31 at RRPV 1).

Results (25M warmup + 25M simulation instructions; speedup is geometric mean over the three traces). **IPC**

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	0.2322	0.2511	0.3821	0.2885
SPP	0.2344	0.2548	0.3873	0.2922
Dynamic IRRIP	0.2320	0.2588	0.3828	0.2912
SPP + Dyn IRRIP (pf-unaware)	0.2341	0.2628	0.3882	0.2951
SPP + Dyn IRRIP (pf-aware)	0.2341	0.2629	0.3883	0.2951
PF insert RRPV 1	0.2340	0.2624	0.3875	0.2947
PF insert RRPV 2	0.2341	0.2625	0.3875	0.2947
PF insert RRPV 3 (final)	0.2341	0.2629	0.3883	0.2951

Speedup over baseline

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	1.0000	1.0000	1.0000	1.0000
SPP	1.0093	1.0148	1.0138	1.0126
Dynamic IRRIP	0.9992	1.0304	1.0019	1.0104
SPP + Dyn IRRIP (pf-unaware)	1.0081	1.0466	1.0162	1.0235
SPP + Dyn IRRIP (pf-aware)	1.0081	1.0467	1.0164	1.0236
PF insert RRPV 1	1.0079	1.0450	1.0143	1.0223
PF insert RRPV 2	1.0082	1.0452	1.0144	1.0225
PF insert RRPV 3 (final)	1.0081	1.0467	1.0164	1.0236

L1D load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	10.679	13.534	3.403	9.205
SPP	9.649	12.900	3.013	8.520
Dynamic IRRIP	10.679	13.544	3.403	9.209
SPP + Dyn IRRIP (pf-unaware)	9.666	12.916	3.017	8.533
SPP + Dyn IRRIP (pf-aware)	9.663	12.916	3.018	8.532
PF insert RRPV 1	9.663	12.915	3.013	8.530
PF insert RRPV 2	9.659	12.912	3.020	8.530
PF insert RRPV 3 (final)	9.663	12.916	3.018	8.532

L1D prefetch accuracy (%)

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	n/a	n/a	n/a	n/a
SPP	52.01	60.02	36.44	49.49
Dynamic IRRIP	n/a	n/a	n/a	n/a
SPP + Dyn IRRIP (pf-unaware)	52.42	59.33	36.08	49.28
SPP + Dyn IRRIP (pf-aware)	52.52	59.89	36.00	49.47
PF insert RRPV 1	52.84	59.75	36.17	49.59
PF insert RRPV 2	52.21	59.59	36.08	49.29
PF insert RRPV 3 (final)	52.52	59.89	36.00	49.47

L1D load coverage (%)

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	0.00	0.00	0.00	0.00
SPP	9.65	4.69	11.48	8.60
Dynamic IRRIP	-0.00	-0.08	0.00	-0.03
SPP + Dyn IRRIP (pf-unaware)	9.49	4.56	11.34	8.46
SPP + Dyn IRRIP (pf-aware)	9.51	4.56	11.32	8.47
PF insert RRPV 1	9.52	4.57	11.47	8.52
PF insert RRPV 2	9.55	4.60	11.27	8.47
PF insert RRPV 3 (final)	9.51	4.56	11.32	8.47

L2C load MPKI

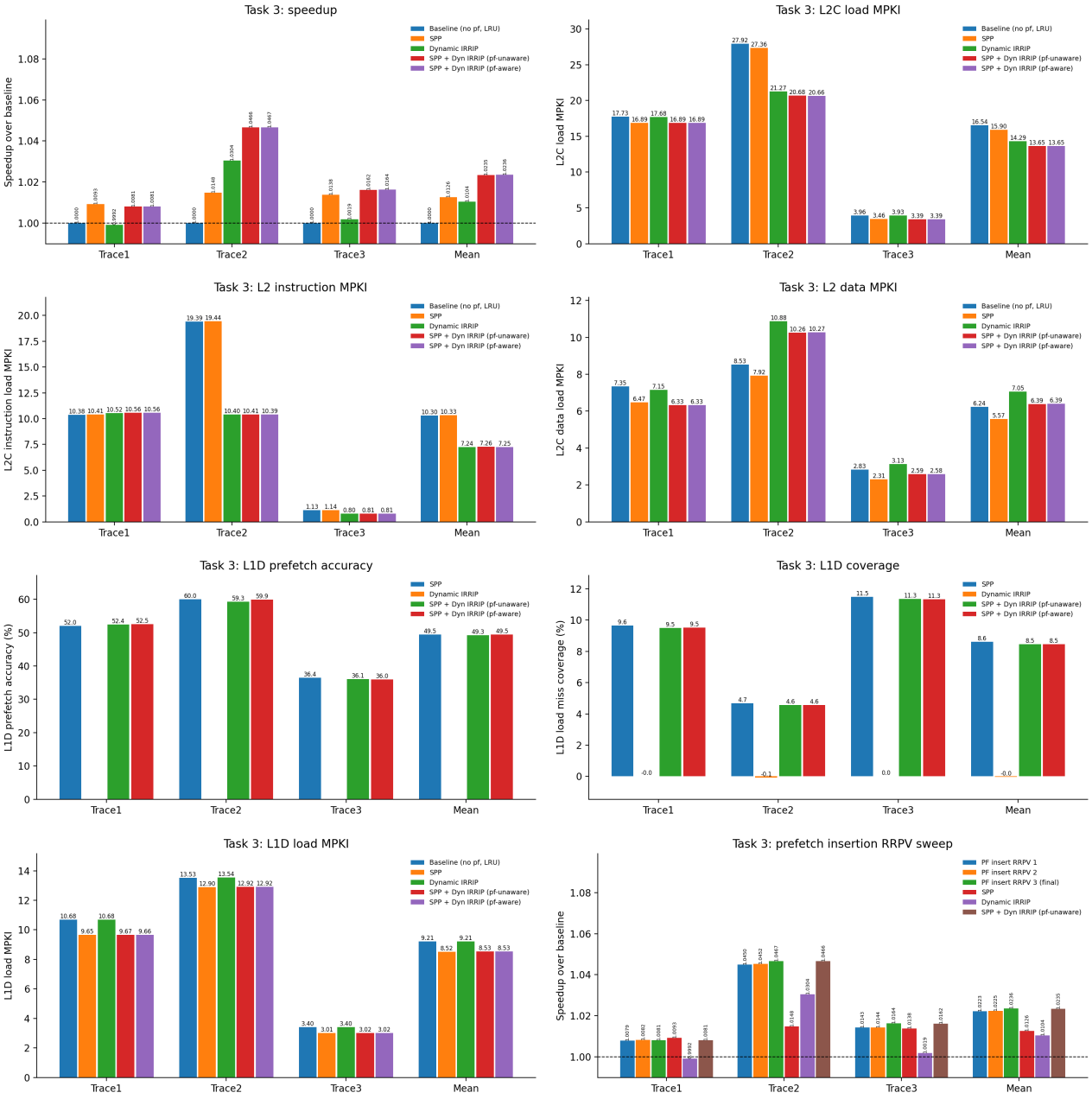
Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	17.730	27.923	3.959	16.538
SPP	16.887	27.361	3.456	15.901
Dynamic IRRIP	17.679	21.272	3.931	14.294
SPP + Dyn IRRIP (pf-unaware)	16.891	20.675	3.393	13.653
SPP + Dyn IRRIP (pf-aware)	16.890	20.663	3.391	13.648
PF insert RRPV 1	16.902	20.749	3.432	13.694
PF insert RRPV 2	16.885	20.739	3.430	13.685
PF insert RRPV 3 (final)	16.890	20.663	3.391	13.648

L2C instruction load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	10.384	19.391	1.126	10.300
SPP	10.414	19.437	1.143	10.331
Dynamic IRRIP	10.525	10.396	0.801	7.241
SPP + Dyn IRRIP (pf-unaware)	10.564	10.411	0.807	7.261
SPP + Dyn IRRIP (pf-aware)	10.563	10.393	0.806	7.254
PF insert RRPV 1	10.574	10.498	0.859	7.310
PF insert RRPV 2	10.570	10.497	0.854	7.307
PF insert RRPV 3 (final)	10.563	10.393	0.806	7.254

L2C data load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	7.346	8.533	2.833	6.237
SPP	6.472	7.924	2.313	5.570
Dynamic IRRIP	7.154	10.876	3.130	7.053
SPP + Dyn IRRIP (pf-unaware)	6.327	10.264	2.586	6.392
SPP + Dyn IRRIP (pf-aware)	6.327	10.270	2.585	6.394
PF insert RRPV 1	6.328	10.251	2.574	6.384
PF insert RRPV 2	6.315	10.242	2.576	6.378
PF insert RRPV 3 (final)	6.327	10.270	2.585	6.394



Findings. SPP + prefetch-aware Dynamic I-RRIP reaches a mean speedup of 1.0236, against 1.0126 for SPP alone and 1.0104 for Dynamic I-RRIP alone, so it outperforms both on the mean and on Traces 2 and 3 (1.0467 and 1.0164). On Trace 1 it does not beat SPP alone (1.0081 versus 1.0093). There the selector chooses SRRIP (final PSEL 8 of 1023), so the prefetch rules are active only in the 32 I-RRIP leader sets, and the combination inherits the small Trace 1 loss of Dynamic I-RRIP (0.9992), which comes from the leader sets that keep running I-RRIP. The gain over the prefetch-unaware policy is 0.0001 in mean speedup, within noise for a single 25M-instruction window: with demand data already inserted at RRPV 3, only the no-promotion rule distinguishes the two. L2 load MPKI is 13.65 for both (16.54 for the baseline, 15.90 for SPP alone), and L1D prefetch accuracy and coverage are essentially unchanged (49.5% / 8.5%), since the change affects L2 residency, not what is prefetched. Most of the improvement over SPP alone comes from the replacement policy: L2 instruction MPKI falls from 10.33 to 7.25.

Limitations

One 25M-instruction window per trace (the simulator is deterministic, but differences of ∼0.1% may not generalise). ChampSim's warmup skips front-end stalls, so the measured region starts with caches and tables relatively cold; this affects all configurations equally. The accuracy counter does not track prefetches that fill L2. ChampSim seeds its random number generator from the trace file name (`src/main.cc`), and Task 2 was run with the traces named `trace1..3` while Tasks 1 and 3 used `Trace1..3`; each task is therefore compared against its own baseline run, and the same policy can differ slightly between tasks (Dynamic I-RRIP: 1.0114 in Task 2B, 1.0104 in Task 3).